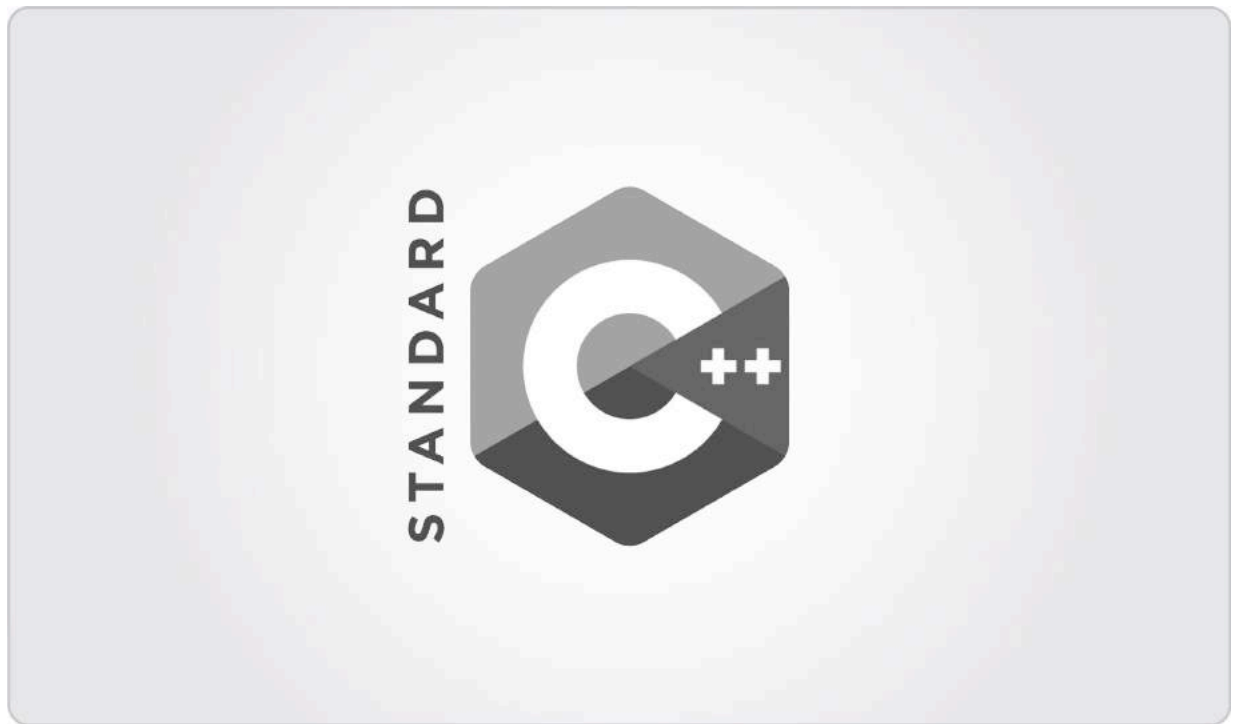


'C' & 'C++' Programming Style Guide

Infostream's rules and recommendations for producing C and C++ programmes



**Infostream Document
Number 001-6001
Revision G**



1 CONTENTS

1	Contents	2
2	Revision History	4
3	Introduction	5
3.1	Formatting	5
3.2	Coding standards and technique	6
3.3	Guiding principles	6
4	Glossary	7
5	File naming	8
5.1	Include files	8
5.2	Source files	8
6	General points	9
6.1	Compiler warnings	9
6.2	The meaning of Index and number	9
6.3	TODO and FIXME	9
6.4	Logo and copyright notices	9
6.5	Existing non conformant code	10
7	Names	11
7.1	General	11
7.2	Variable names	11
7.3	C++ argument names	12
7.4	C++ interfaces names	12
7.5	C++ exception names	13
7.6	Camel case vs. underscores	13
8	Formatting style	14
8.1	Braces	14
8.2	Parenthesis	14
8.3	Indenting	15
8.4	Functional blocks	15
8.5	Use of whitespace	15
8.6	Bracketing blocks	16
8.7	Indenting	16
8.8	Alignment	16
9	Coding standards	18
9.1	Introduction	18
9.2	Redundant/Unreachable code	18
9.3	Falling through switch statements	18
9.4	Variable arguments	18
9.5	Clarify order of operation with parenthesis	19
9.6	Virtual functions	19
9.7	Cryptic expressions	19
9.8	Macros	20
9.9	Types	20
9.10	Use of const	21
9.11	Comments	22
9.12	Variables	23

9.13	Functions	23
9.14	Boiler plate members	24
9.15	Default arguments	25
9.16	Exceptions	25
9.17	Exception logging	25
9.18	Too broad exception clause	25
9.19	Initialisation	26
9.20	Conversion and conversion operators	26
9.21	Use explicit for constructors	26
9.22	Use templates wisely	26
9.23	Virtual destructor	26
9.24	Multiple inheritance	26
9.25	Prefer composition to inheritance.	26
9.26	Friendship	26
9.27	Keep private data private	27
9.28	RTTI	27
9.29	Casting	27
9.30	Anonymous namespaces	27
9.31	Make use of RAI	27
9.32	Memory ownership	27
9.33	Private, protected, then public	27
9.34	Abstract base classes	28
9.35	Variables	28
9.36	Incrementing	28
9.37	Commenting classes	28
9.38	Operator overloading	28
9.39	No 'using' in headers	29
10	Idioms	30
10.1	Forever loops	30
10.2	Simple loops from zero	30

2 REVISION HISTORY

Revision	Date	Notes
A	31st Jan 2000	Initial release
B	15th Feb 2000	Use of signed and unsigned chars for holding numeric values
C	6th Sep 2005	Introduced document into electronic revision control system.
D	17th Oct 2014	Substantially updated and expanded Incorporated C++11 changes.
E	4th Nov 2014	Added reference to C++ Coding Standards book.
F	16th Feb 2016	Added copyright banner examples. Clarified rules regarding const declarations. Reformatted code examples to Menlo font. Added section for idioms. Updated document style, formatting and fonts
G	8th Sep 2016	Typos mostly. Expanded glossary and idioms Added remarks on dealing with legacy inconsistent code Tiny improvements to banner ASCII art

3 INTRODUCTION

“Programs must be written for people to read, and only incidentally for machines to execute”

- Harold Abelson, *Structure and Interpretation of Computer Programs*.

This document is Infostream’s C/C++ style guide.

There are two aspects to programming styles guides – the first relates to the way the code is formatted (how it looks), and the second relates to how the code is structured. Some style guides focus on one aspect and not the other however for Infostream, both are important.

No programming style and coding guidelines will solve all problems, nor will they remain static indefinitely. As the languages used improve, the style guide should be updated to take advantage of better ways of doing things. This latest revision of the document has some changes arising from the introduction of C++11. Doubtless there is plenty of scope for improvement.

style

noun

- 1 A particular procedure by which something is done; a manner or way. A way of painting, writing, composing, building, etc., characteristic of a particular period, place, person, or movement. A way of using language. One’s usual way of behaving or approaching situations.
- 2 A distinctive appearance, typically determined by the principles according to which something is designed.

3.1 Formatting

There are many arguments for and against rigid rules regarding the formatting of the source code text. Infostream’s position is that code is already difficult enough to read, that anything that can be done to make it simpler should be employed. Even little annoyances (positions of braces etc) are all avoidable distractions that make the logic of the code harder to follow.

Consistency matters. Just as in spelling and except for some historical arguments between the US and the UK, there is no doubt about the correct way to spell English words. After we finish grade school, we all learn that there is only one way (whether we agree or not) to spell any given word. While there might be alternative and perhaps more sensible ways to spell certain words, when writing the language, the spelling convention is universally adopted. It is the same with coding standards. The standard is always subject to discussion, but adherence to the standard isn’t.

At the time of this update, we are investigating the use of automatic code formatters to slightly reduce the need to define a fixed set of formatting rules, but as at the time of writing, none have been adopted. Any checked in code must therefore follow the rules below.

For a software engineer, the code on the screen is the thing they look at for more hours in the day than any thing else. For the same reasons that apply for dress standards and maintenance of a clean and tidy work environment, code should be formatted consistently, cleanly and attractively.

You should aim for “beautiful” code. Appearance matters!

The Infostream formatting standards are quite similar in many respects to the Google C++ Style Guide. If aspects of code formatting are not covered specifically in this guide, the Google C++ guide can be useful.

3.2 Coding standards and technique

Any language can be written well or poorly. While some languages support more structured code, with enough imagination a determined (and bad) programmer can produce terrible software no matter how hard the language tries to protect against it. Similarly, excellent software can be written with even the least restrictive of all languages (i.e. assembly code).

In the C++ context, there are some excellent resources available to improve the readability and reliability of code.

Some of the rules in this guide related to writing code that is more likely to be correct when run. However many of them, if not the majority, will assist the programmer (and future programmers) in reading the code. That is, the rule is there to define how someone other than the original programmer reading the code should interpret the source code.

One of the best books on the subject of C++ coding standards is C++ Coding Standards 101 Rules, Guidelines, and Best Practices (C++ in Depth Series) written by Herb Sutter and Andrei Alexandrescu. 90% of the rules and guidelines in that book are, or ought to be adopted by Infostream. Those rules are not reproduced here but should be read and understood.

3.3 Guiding principles

- Follow the path of least surprise. Use the expressive power of the language to make your intention clear – not just to the computer but to the programmer that follows you. Where there is more than one way to do something, pick the method that is clearer. Where there is a trade off between clarity and efficiency – clarity should win 99% of the time.
- Re-factor aggressively. Writing beautiful code is an iterative process. There are few artisans in any field that can create a work of art in a single stroke of the pen or brush. Similarly for software. Your first line of code is not precious. Re-write it, overwrite it, and replace it as necessary on the path to the final result.
- Understandable code is much more useful than documentation.
- Do not excessively optimize for speed over clarity

4 GLOSSARY

Non-obvious terms or special usage, or unique interpretations of common words should be defined here.

Camel case	The practice of using capital letters to denote the start of words within a file/function/variable name. We don't use it.
Index	An explicit type of number that is intended to reference a member of an array (or similar). We interpret index to mean a value that starts at zero (for the first element). The term "Zero based Index" is a tautology
RTTI	Runtime Type Information is the concept of determining the type of any variable during execution (runtime.) The RTTI mechanism contains: The operator <code>dynamic_cast</code> . The operator <code>typeid</code> . The struct <code>type_info</code> .
RAII	Resource Acquisition Is Initialization is a programming idiom used in several object-oriented languages, most prominently C++ In RAII, holding a resource is tied to object lifetime: resource allocation (acquisition) is done during object creation (specifically initialization), by the constructor, while resource deallocation (release) is done during object destruction, by the destructor. If objects are destroyed properly, resource leaks do not occur

5 FILE NAMING

5.1 Include files

Sometimes programmers try to pretty up the code by camel casing file names. Don't. An example would be 'StdIO.h' or 'StdLib.h'. This is wrong. These files are actually called "stdio.h" and "stdlib.h" written exactly that way and have been called that since K&R. MSDOS (and windows) may not object about the capitalisation but it is not portable. Don't use it.

5.2 Source files

Most decent operating systems tend to use simple lower case letters for file names by default. Certainly, they are easier to type. For portability reasons, it is best to ignore the case of file names by the simple expedient of using lower case exclusively. Mixing case (as in the include file example above) is clumsy to type even (or perhaps even more so) for experienced touch typists.

Avoid anything that makes file name typing more difficult and less portable. Underscores, capital letters, punctuation are all generally a bad idea. Stick to straight out alphanumeric, lower case file names.

Don't make them excessively long. You may use an IDE with point and click access to the source file. Some programmers actually type the names! Auto-completion is a commonly, but it is helpful if the names are different after the first few letters so that auto-complete can do its work.

6 GENERAL POINTS

6.1 Compiler warnings

There is almost no compiler generated warning that is not an actually or potential bug. Don't ignore the warnings. Certainly, don't submit code that generates warnings. Apart from the point above, any warning that is ignored on the assumption that it is harmless, tends to hide other (later introduced) warnings that may not be. Don't think to yourself "I know what that warning is, I'll fix it up later". It will still hide other important warnings.

6.2 The meaning of Index and number

The terms "index" and "number" should be used carefully and specifically. An index is defined as a value that goes from zero to some positive upper limit, such as would be used to access an array. A number is something that is not an index.

By defining these terms in this precise way, you avoid the expression "zero based number" or equally awkward equivalents.

If you have an array of objects and you want to refer to them with a variable – use the word index in the variable name rather than number – for an array of "widgets" use `widget_index` rather than `widget_number`.

6.3 TODO and FIXME

Infostream's methodology is substantially 'Agile' and iterative. The first version of a program should be a "complete" version, with all executions paths resolved and with no logical errors. Thus, every project starts with a "hello world" version, and iteratively approaches the final result with progressively more functionality along the way.

Where there is known incomplete behaviour, always mark the spot with TODO.

Known bugs should be marked with FIXME.

Almost all editing environments have special features to handle these character sequences to make them easier to find in the code.

6.4 Logo and copyright notices

6.4.1 Source files (compilation units)

The following banner text should be added to the top of every source file but NOT the header files. A similar style header should be applied to source files in other languages, using the appropriate comment delimiters as required.

$\ast/$

7 NAMES

7.1 General

Names should be descriptive of the value of a variable, never of its type. In particular and specifically, the so-called “Hungarian notation” which describes in a particular code the functions exact return type or a variables type is not acceptable practice. The only time where it might conceivably be used, is where a temporary variable is created or returned specifically to interface to an operating system call which is documented using this style and then only if it makes it clear that the variable’s scope is to serve that specific purpose.

It is pointless (and often counter-productive) to prefix a string with the letters ‘sz’. This is the ‘C’ language, strings are by definition zero terminated and there is no need to remind the programmer of the fact. Or ‘csz’ for a constant version of the same.

For C++ this is even more so, because the language provides the flexibility to sub-class a built in type or construct entirely new types (sub-range checked integers or smart strings or pointers). Marking a variable type in the name would require renaming the variable or risk having the name be misleading as to the type.

Don’t worry, the compiler will tell you quickly enough if you try to assign something to this constant. (See “aggressive use of const” below).

Naming variables based on the type reduces program clarity rather than enhances it. The program should be coded in a type safe manner in any case and the type should for the most part be irrelevant at the time of use. In any case, most variables live with a scope of less than 20 lines and the declaration of the function should be immediately visible at the point it is used.

C++ has been designed so that you can create proper types that are indistinguishable from the built-in types. That was an explicitly stated design goal. It often happens that a variable that starts out as a simple built in type (e.g. an integer) and later morphs into a more complex type. The name of the variable should not have to change because of the change of type.

Using some single letter shortcuts as a prefix or postfix to the name is ok. An example would be “f_” which can be used to mean “flag indicating the state of” and then whatever the flag is for. Note that this is not the same as “b_” to mean Boolean, which is a type description and is thus not acceptable. “f_” is a purpose description and basically a common and widely understood abbreviation for an otherwise acceptable name.

Note: More common Infostream practice is to use an underscore/letter postfix.

Names should lower case words, abbreviated if the meaning remains clear, separated by underscores.

Using p_ to mean pointer is a tricky one. Pointers are not really types as such, so this does not constitute Hungarian notation. Its probably ok.

7.2 Variable names

Variable names should in general, describe the value they hold. However sometimes where variables don’t specifically mean anything, other than to say iterate through a loop, and nothing else, then use a single letter name such as ‘i’. In this context, it means “iterator” and the name is simply a shorthand form. Similarly, use ‘s’ to mean “a string” where the purpose and use of the string is obvious from the context. This particularly applies to temporary or intermediate variables.

Short names are often ok when the value is clear from the context and especially where the scope is (very) limited.

However, be careful not to reuse short names within nested scopes. This is one place where it is very easy to get confused as to just exactly which version of the variable 'i' you mean.

The compiler doesn't care what the variable names are – other than each variable name is unique. Therefore you must, by definition, be naming the variable for the benefit of the person reading the source code, not for the benefit of the compiler. Otherwise you would call the first variable a, the next b, and so on. So try hard to pick the best possible name for the variable – and that means the name that makes it the clearest as to what is going on. Don't settle for a second best name.

Don't use slight variations on a name to distinguish between a local and a global or a member variable and a member function argument. The absolutely worst example of this (and frustratingly common) is to use a capitalization change to distinguish names. This is the most appalling source of bugs. Don't contract, post or prefix is numbers or single letters etc.

7.3 C++ argument names

In C++, especially in assignment type functions, the function argument name might usefully be named exactly the same as the class member. This is perfectly acceptable – the scope resolution operator makes the meaning clear.

```
class my_class {
    int width;
...
public:
    set_width(int width)
    { my_class::width = width; } /* clear and simple */
}

class my_class {
    int width;
...
public:
    set_width(int width)
    { this->width = width; } /* clear and simple */
}
```

Sometimes you see this sort of thing handled by using a prefix to the argument name such as a `_width`. Completely unnecessary and much less clear. If the variable is intended to be the exact same thing, give it the exact same name.

Similar to the above, an initializer list for a constructor has name disambiguation built-in by context.

```
class my_class {
    int const width;
...
public:
    my_class(int width) :
        width(width) /* explicitly clear */
    { ... }
}
```

7.4 C++ interfaces names

Postfix C++ pure abstract classes with `_i` (interface). This improves code clarity.

7.5 C++ exception names

Postfix exception classes with the 'exception' suffix.

Internal note: Consider revising this guide to propose `_e` for exceptions. Enums are types, and therefore would use `_t`

7.6 Camel case vs. underscores

We have chosen to strongly prefer the use of variable and function names that are all lower case with words separated by underscores. Since legibility is key objective, the separation of words within a name via underscores is more equivalent to the conventional separation of words with whitespace.

Since camel case and underscores are roughly equivalent in terms of typing (in)convenience, neither wins on the basis of code entry.

Camel case is a very artificial and unnatural (and therefore less readable) method of separating words within a variable name.

8 FORMATTING STYLE

8.1 Braces

In general, try to format the code in such a way as to reduce the number of lines taken to express a statement, without unduly impairing the readability. This will assist in keeping the length of functions short. Very short conditional statements can perfectly well be written in-line.

```
if (condition) function2(); else function2();
```

This is just as clear (and only takes one line) when compared to:

```
if (condition)
    function2();
else
    function2();
or even worse:
```

```
if (condition)
{
    function2();
}
else
{
    function2();
}
```

which is just a waste of lines.

8.1.1 Around functions

About the only time you should not do this is the braces surrounding a function. These should always be on a single line by themselves.

8.2 Parenthesis

Use more parentheses rather than less. 'C' has some funny precedence so use brackets to eliminate any possible ambiguity.

Always use parenthesis around the conditional when using the conditional operator (a?b:c). The precedence's can cause problems and the operator can often be difficult to read.

8.2.1 Example

```
int const max((a > b) ? a : b);
```

8.3 Indenting

```
switch (expression) {
case 1: /* expression */ break;
case 2:
    /* expression */
    break;
} /* switch */
```

If you decide to put statements in a case all on the same line, make sure all will fit on the same line. Don't put some on the same line, and others on subsequent lines. Anyone looking over the code will tend to notice that it is concatenated into a single line, or that each statement is one line at a time. If you mix it up, you get mixed visual cues and it makes it harder to follow the code.

Sometimes in switch statements, programmers use additional braces within a single case. Often this is done to introduce a local variable within that case. Don't do this. If you want a local variable, the statements within the case almost always would make a very nice little function instead.

8.4 Functional blocks

Sometimes, to avoid the proliferation of source files, several blocks of related code is incorporated within a single file (compilation unit). In this case, each block should be separated with a very clear comment comprising several lines, with a simple one-line descriptor for the block. For example

```
/******
LCD driver functions
******/
```

By placing functions into blocks like this, it becomes a trivial matter to split files into smaller ones once the file becomes too big to be manageable.

8.5 Use of whitespace

Do place spaces after and before keywords:

```
do<space>{ statements }<space>while<space>(expression);
```

Do place spaces between tokens in arithmetic expressions

```
int a<space>=<space>b<space>+<space>c;
```

Do not put spaces between parenthesis and the enclosed expression but

Do put spaces on the outside of parenthesis except

Do not put a space between the function name and the parenthesis enclosing the arguments

If writing several statements on a single line, separate each statement by a space, immediately after the semi-colon.

8.6 Bracketing blocks

8.6.1 Switch statements

```
switch (i) {  
case 1: ... break;  
case 2: ... break;  
default: ... break;  
}
```

8.6.2 Conditional statements

```
if (condition) {  
    ...  
} else if (...) {  
    ...  
} else {  
    ...  
}
```

8.7 Indenting

Each nested block should be indented four characters relative to the outer one. Tabs, by ancient convention and almost universally are aligned to even eight character columns. Don't change your editor to use 4 character tab stops – when you list in DOS or otherwise, or print the file – mostly it will assume 8 character tabs and the result will be unreadable.

If your editor has an option to replace tabs with spaces, then you may set the editor to 4 character tabs for the purpose of generating source code. However, other code will be generated with 8 character tabs and when viewing those files, you will see unreadable code. Even worse, if it then substitutes the tabs with spaces and you write it back, the file will be ruined for everyone else. So again, eight character tabs are recommended. Most decent editors support auto indenting so you don't often have to type the four spaces necessary to indent.

8.8 Alignment

Unless it really helps to make a particular meaning clear, don't align columns. For example if defining a set of numbers use a single space between the keywords and numbers like this:

```
#define VALUE_NORMAL 10  
#define VALUE_EXCEPTIONAL 11  
#define VALUE_ERROR 12
```

rather than like this:

```
#define VALUE_NORMAL      10  
#define VALUE_EXCEPTIONAL 11  
#define VALUE_ERROR      12
```

The above example is line-ball. A worse (and typical example) is in the code:


```
Variable_1          = 5 * (a + b) / 3;  
Longer_variable_name = (c + d) / 3;
```

This is silly and does nothing to improve the readability of the code.

9 CODING STANDARDS

9.1 Introduction

It's difficult to come up with general rules about what is better program logic than something else is. However, in writing, consider techniques that reduce the damage caused if an unexpected condition occurs. The program may (and probably will) be modified in the future (and probably not by you) and the following programmer may not understand the assumptions you have made in handling a limited number of possible cases.

Consider the following simple examples.

- In loop terminators, use over-run proof expressions. For example, if a loop goes from zero to 9, terminate with $(x < 10)$ rather than $(x != 10)$. That way, even if the manages to somehow skip over the terminating value, the loop will still stop – hopefully without too much damage being done.
- If using a switch to process a value, even if the application can not generate cases other than those which the switch anticipates, consider coding in a way that is safe even if an unexpected value is received. This will make a more robust program – should it ever be changed in the future.
- Don't maintain two copies of the same value, or have a single data be represented in two or more different ways. Firstly, this is inefficient, because both data sets need to be maintained. Secondly, it is very easy to make the mistake of maintaining only one, and forget the other. If you must do this, (and the need for extreme speed would be one of the few reason to do so) try to do it in such a way that the data is self-correcting. For example compute a new value based on one copy of the data, and write the results twice. This would be preferable to computing the two new values independently, since once out of sync, they remain that way.

9.2 Redundant/Unreachable code

This will normally generate a warning and that is covered elsewhere. However, even if it doesn't, avoid doing it. Don't for example place a break after a return in a switch statement.

9.3 Falling through switch statements

'C' will let you fall through a case in a switch statement. It is normally poor practice. If you see code like this, you are more likely to assume that the code is wrong, than it was intended that way. If you must do it, put a comment to say that is what you meant. Better still, see if you can write the code so that the common bit is written as a function. You will probably end up with easier to understand code that way.

9.4 Variable arguments

In general, passing arguments to functions that are modified by the function is discouraged. Functions preferentially should use arguments for input and return values for output.

Where the above is impractical (too many values by the modified by the function for example, any variable argument that is passed to the function which is expected to change should be passed as a pointer, not a reference.

The use of a pointer makes it clear to the reader that the programmer of the function intends to modify the arguments.

Don't use references for argument that are intended to be changed. Use pointer, and thus require the & to make it obvious.

9.5 Clarify order of operation with parenthesis

Although the order of operation in C++ is well defined, we strongly encourage the use of parenthesis to make the order clear. This is particularly true for complex multi-part logical expressions where parenthesis should always be placed around separate logical values.

9.6 Virtual functions

9.6.1 Override keyword

The override keyword was introduced in C++11 to force a compile time error for a common coding mistake that was previously only detectable during run-time. Its use is mandatory (not optional), where appropriate, on any new code and it should be added semi-aggressively when editing legacy code written in an earlier C++ dialect.

9.6.2 Final keyword

Also introduced in C++11. The final keyword should only be used when you wish to explicitly prevent a virtual function override for a public or protected member. It is not required for a private function, since making the virtual function private implies final. It is discouraged on private virtual function overrides.

9.6.3 Redundant use of “virtual” keyword

The virtual keyword is redundant on an overriding function. Prior to C++11, the programmer could only use this redundant keyword to signal to the reader that the derived class virtual function was an override. Since C++11, the use of the keyword “override” serves this purpose and the optional virtual keyword is no longer encouraged when the more explicit alternative is available.

Older rule (not applicable to C++11). When deriving from base classes containing virtual methods make the methods virtual in the derived class as well. For code clarity and readability make it obvious that a method is virtual.

9.6.4 Virtual functions in constructors and destructors

Calling virtual methods when an object is being created or destroyed leads to undefined behaviour.

9.7 Cryptic expressions

‘C’ is a cryptic language in many ways or, from an alternative viewpoint, it is syntactically efficient. If you can do something with less code, then feel free to do it that way. If absolutely necessary, put a comment to clarify the purpose – but never to clarify the syntax. An experienced ‘C’ programmer will understand the meaning much more clearly from the language itself than a (possibly inaccurate or ambiguous) comment.

9.7.1 Loop over range of integers (where order doesn't matter)

```
for (int i = 10; i--;) { ... }
```

9.7.2 Conditional expressions

```
return (condition) ? value1 : value2;
```

rather than:

```
if (condition) return value1; else return value2;
```

or, better still if the return values are true/false:

```
return condition;
```

or

```
return !condition;
```

9.7.3 Infinite loops

The preferred implementation for an infinite loop is:

```
for (;;) {  
    if (/* some condition */) break;  
}
```

9.8 Macros

In C++, the usual rule of not using macros unless absolutely necessary should be applied rigorously. Macros are processed as part of the pre-processor, not by the compiler. This makes their use potentially error prone in a C++ context. Use const variables or enumerated types and inline functions to achieve the same ends as macros do in C code.

Macros should be named using upper case letters but other than this are named in exactly the same way as for functions and variables or constants.

Always always always, put parenthesis around the macro definition and around any parameters of the macro. Like this:

```
#define MY_MACRO(ARG) ((ARG) * 2)
```

9.9 Types

9.9.1 Use of void ('C' specific)

Always use 'void' where possible. Functions that do not return a value must be declared void. In 'C' an empty argument list should be void (so that it prototypes itself) but in 'C++' an empty argument list does the same job and it is incorrect to use void in this case.

9.9.2 Typedefs

Typedef should be used only to define a legitimately new type, not to give shorthand names to built-in types. The type of an unsigned long variable is just “unsigned long”. It is not a ULONG or, even worse, a DWORD.

It is often useful, especially where you create a single use type to give the type-name the postfix ‘_t’ to say that it is a type rather than an instance of the type. For example

```
/* define a new type */
typedef item_iterator_t double_list_iterator<item_t>
...
/* define a single item of this type of the same name */
item_iterator_t item_iterator(item_list);
```

This saves coming up with new names for the item and the type.

9.9.3 Signed and unsigned chars

When using chars to hold numeric values (as opposed to actual characters) always explicitly qualify the type as either signed or unsigned. The language does not define whether chars are signed or unsigned by default. In any case, it is a useful cue that the value is numeric rather than just a character.

There is often not much reason to use such small integer types, and it is generally easier (and no less efficient) to use shorts. Indeed on some processors, it might be less efficient (from a code standpoint) to use chars. However on most 8 bit processors, chars offer significant savings, especially if the compiler implements function argument and return value passing via registers.

9.10 Use of const

Use const everywhere it can be used. If a string is not modified, it is const. If an int is not intended to be modified, it is also ‘const’. Marking a value as a constant sends a powerful signal to the reader regarding the intent of the value.

In C++ apply const to stack variables, methods, member variables and method arguments wherever this can be achieved.

If you intend a variable to be unmodified, explicitly declare it const.

9.10.1.1 Good

```
char const* const s(“hello world”);
```

9.10.1.2 Bad

```
char* s = “hello world”;
```

9.10.2 type const v.s. const type

Note the example above where the constant string is declared char const* rather than const char* (which is, admittedly, much more common). However, the Infostream rule is to use the first form for consistency as explained below.

Consider the following:

```
const char* p = "hello world"
char const* p = "hello world"
```

Both are correct and valid in C++, but what is constant?

In the examples above, it is the characters "hello world" that are constant. You can change the value of the variable 'p', but you can't change the characters in the string "hello world". This is simple, familiar enough and very common. Therefore it follows that:

```
p = "new string"
```

would be legal using the example above, but

```
p[0] = 'H';
```

will generate a compile error. To make the first case illegal (that is, to make 'p' itself constant), you must use

```
char const* const p("hello world").
```

Or, if you want a string that you can change:

```
char* const p(new char[10]);
```

Notice that the ONLY place you can put the 'const' keyword in order to make 'p' itself constant is AFTER the declaration of its type const char* or char* in this case) and immediately before the variable name. In the example above, note that you can copy into the string (to change it), but you can't change p. Therefore:

```
delete p;
```

will always be correct – which is extremely useful. Therefore if we follow the same rules for all other variable types we get

```
int const n(10).
```

So, for consistency and clarity, the const declaration is always placed after the type just before the variable name, regardless whether it is a pointer, integer or user defined type. Many text books and examples don't use this construction, but then most text books and examples don't rigorously promote the aggressive use of "const-ness". The rule here is "if you don't intend to something will change, it must be const". Use the language itself to document the intent! Consider...

```
char* p(new char[10]); /* always ensure that p never changes! */
```

```
...
delete p;
```

vs.

```
char* const p(new char[10]) /* the above comment is superfluous */
```

9.11 Comments

Comments are used to make the intention of the code clearer than the code itself can be. Thus it follows that a comment is appropriate when the code is itself not sufficiently clear. This should be your guide. A comment should be regarded as a failure to make the code itself clear.

The source code itself is the final determinant of what the program will do and is thus the definitive copy of the documentation. The job of the programmer is to write the code in a way that is both correct for the computer to interpret (and therefore run correctly) and easy for the code reviewer to interpret.

Any additional documentation (including comments) is by definition a redundant copy and therefore to be strongly avoided if possible for all of the oft repeated reasons.

There are very few good arguments that can be made to prefer a comment over writing the code itself in a self explanatory manner.

For example, if writing a loop, placing the entire loop body within a function provides the opportunity to give the function and therefore the loop body, a descriptive name. The obvious benefits of making the loop parameters much more explicit is the use of function arguments and return values.

Comments are very appropriate when something unusual is happening in the code that cannot be made obvious by the structure of the code itself. However that unusualness should be a warning to the programmer with a strong preference to eliminate what is unusual

When something unusual is happening, make it clear.

Comments are, when it comes to explaining program flow, pretty much a waste of time if the code is well written.

Comments are useful to describe what a function is intended to do.

Comments are useful at the end of a large block, to indicate which block is being closed.

```
for (;;) {  
    switch (get_key()) {  
        case ESCAPE: return -1;  
        default: process_key(); break;  
    } /* switch */  
} /* forever */
```

9.12 Variables

Put global variables at the top of the file, after the header file inclusions but before any functions.

9.13 Functions

9.13.1 Use of static

There are only two acceptable types of functions - ones that are static or ones that are used globally. If they are not used globally they must be declared static. Even if you write a function and think it might one day be used globally but are not immediately intending to do so, declare it static.

9.13.2 Length

Functions should be kept as short as possible. A function becomes readable and more importantly its operation becomes clearer the shorter it gets. The ability to assess a function as being bug free increases dramatically if it can be viewed at a single glance. Therefore as a general rule, no function should be longer than 20 to 25 lines.

9.13.3 Delineation

Every function should be clearly delineated from the others. This should be done with a clearly visible comment line spaced one line above the function name. The comment should contain a 1-line description of the operation of the function.

Example:

```
/* ---- copy source string to dest string returning pointer to dest ---- */  
  
char* strcpy(char* dest, const char* source)  
{  
    ...  
}
```

Important note: Format the code in this way as you work – don't go through it later and fix it up. The comments that you write at the top of each function define what the function will do – getting that clear in your mind by writing a comment to the effect is good practice anyway. Also, if you want someone to look over your shoulder at a problem you are having, its much easier for them to look at a properly formatted job in progress.

9.13.4 Ordering

Functions should be ordered so that all function calls are made upwards within the file. This implies that main() is always the last function in the file.

9.13.5 Naming

Functions should be given descriptive names. The description should relate to the return value or operation being performed. If the operation of the function cannot be usefully described, consider reducing the length of the function and hence the number of operations it performs.

It is often useful to make the function name, especially when returning a true/false value which is used in a conditional expression, read like a question for which it (the function) supplies an answer. For example:

```
int is_button_pressed(int button_index) {  
    return (button_values[button_index] == BUTTON_PRESSED);  
}  
  
if (is_button_pressed(MENU_BUTTON)) { ... }
```

In all other respects, names should follow the conventions outlined elsewhere.

9.13.6 Bracketing

Open brackets should be placed by themselves on the line immediately below the function declaration. The close bracket is by itself as the last line of the function. An empty line should always follow the end of the function.

9.14 Boiler place members

Avoid using the default boilerplate member functions: constructor, destructor, copy constructor and assignment

These methods either need to be declared and implemented as public methods or if a class is not copyable or destructible some of the methods must be declared as private.

Mostly it is sufficient to make the private implementation an empty method.

9.14.1 Deep vs. shallow copy

The current C++ (vs. ancient version), provide reasonable (deep) copy behaviour in many circumstances. The most common shallow copy problem relates to the use of naked pointers to dynamically allocated memory. Such pointers provide no memory management functions or reference counting and are extremely prone to errors (regardless of the shallow copy problem).

There are two recommended strategies for dealing with shallow copying problems in C++;

9.14.1.1 Avoid naked pointers

This is good practice anyway. Using smart pointers throughout the code dramatically decreases the problem of memory leaks and completely avoids the pointer-copying problem when pointers are used in classes.

The default compiler copy semantics will apply the copy constructor to any member with copy semantics defined. So strings and smart pointers will copy correctly whereas naked pointers will not.

9.14.1.2 Explicitly define copy semantics

If a class is complex and a regular shallow copy (default compiler copy constructor) will not work, explicitly declare the copy constructor and assignment operator as a private member in the class whether you provide the functionality or not. Declaring a function doesn't reference it, and by doing so, you suppress the compiler defaults and prevent any unintended use of such constructors by the unwary.

It is good practice to define (private) assignment and copy constructors on any class that you suspect will not shallow copy properly, whether you intend (today) that the class will be used in that way or not. Be defensive in your programming style.

C++11 provides for explicit defaults and deleted member functions (including constructors). Use these preferentially to the above if possible.

9.15 Default arguments

The use of C++ default arguments for function parameters is acceptable barring where this causes confusion. For example, function pointer signatures would have to match those of the function being pointed to with defaulted arguments.

9.16 Exceptions

Exceptions may be used but only to indicate exceptional circumstances. Using an exception to control program flow under normal operation is not acceptable.

Be aware that an exception causes an early return from a method and may cause the stack to unwind past a number of stack frames. Pointers stored on the stack should therefore be 'smart pointers' such as `shared_ptr` to allow resources to be released properly and reclaimed. See 9.32

9.17 Exception logging

Log exceptions when they are caught. Logging these changes of program flow allows problems to be identified and resolved. The exception and log entry should alert the system user to the exceptional condition that has occurred.

9.18 Too broad exception clause

If an exception clause is too broad the exception is no longer meaningful. Catch specific exceptions and take specific actions based on that exception.

9.19 Initialisation

Prefer to use initialisation to assignment. Initialisation is generally more efficient than assignment that involves operating on a constructed object. Do this for objects created on the stack as well as when initialising member variables of a class in a constructor initialiser list.

9.20 Conversion and conversion operators

Do not define conversion operators within a class. Classes that define conversion operators allow an implicit conversion to be performed. An unexpected conversion can lead to coding errors.

C++11 – New language features allow much tighter control over the use of conversion operators and substantially solve this problem. Consider revising this rule.

9.21 Use explicit for constructors

Constructors that take a single argument can cause implicit conversion to occur. Use explicit on constructors to prevent this.

9.22 Use templates wisely

Templates are a powerful tool for writing generic code. Their usage is encouraged, however avoid using templates in overly complicated ways as this reduces code clarity and therefore can become the source of bugs.

9.23 Virtual destructor

When deriving a class from a base class, ensure that the base class's destructor is virtual. If this rule is not followed it is possible to partially destroy an object when the delete operator is used on the subclass.

9.24 Multiple inheritance

Avoid using multiple inheritance except where it makes sense.

Multiple inheritance is useful when inheriting multiple abstract interface definitions for example. Multiple inheritance of implementation is rarely useful and the goal can be accomplished through other means such as through composition.

9.25 Prefer composition to inheritance.

If inheritance is for an is-a case then inheritance is fine. Also use inheritance for an interface definition from an abstract base class. For has-a relationships, use composition.

9.26 Friendship

Use friendship in appropriate contexts such as for unit tests.

In the interests of encapsulation and decoupling a class should be accessible through its public interface. On occasion two or more classes are designed to work very tightly together such as iterators and collections over which they iterate and unit tests that test a class's implementation. In these cases the classes need full access to another class and friendship is to be allowed.

9.27 Keep private data private

Do not provide accessor methods for private members held within the class. Encapsulation ensures that the design is not tightly coupled allowing a class's implementation to be reworked with minimal effect to other classes and subsystems. This level of decoupling promotes good design.

9.28 RTTI (Run-time type information)

Do not use RTTI and avoid using `dynamic_cast`.

The need to use RTTI and dynamic casting is indicative of a problem with the code's design. Consider redesigning your code to remove these.

9.29 Casting

Use C++ style casts `const_cast`, `static_cast` and `reinterpret_cast`.

C style casts can cause conversion to occur which may be unsafe. Using the C++ style casts is safer. Using `dynamic_cast` is indicative of a code design issue and needs to be addressed through redesigning without `dynamic_cast` and RTTI

9.30 Anonymous namespaces

Make use of anonymous namespaces for module data and module helper functions.

Prevent name clashes by making use of anonymous namespaces. A compilation module should contain an anonymous namespace for functions and data that are used within the module.

9.31 Make use of RAII (Resource Acquisition Is Initialisation)

Using RAII produces less error prone code. Exception management is made easier and the code is more readable. RAII is useful to manage resource deallocation needed when an exception is thrown.

9.32 Memory ownership

Retain ownership of pointers with the class that performed the allocation.

Memory allocation that is performed within a class should be tracked by pointers within the class. This provides clear ownership of the objects. If ownership is to be transferred, `shared_ptr` should be used. The shared pointer is then the owner of the object pointed to. With newer versions of the compiler other pointers are introduced into the standard such as `weak_ptr`. These can be used to transfer ownership too.

9.33 Private, protected, then public

Struct and class are identical except that struct is public by default and class is private by default. Conventionally we use class for 'proper' classes but it is perfectly acceptable to use struct.

We follow the convention of declaration by definition for regular C and avoid the use of declarations. In C++ of course, declarations are mandatory but should follow the same ordering. Private members are called only by other members of the class (including the public members) so they are placed first. Protected second and public last. The public members, like the "main" of a C program are the entry point for 3rd parties.

It is strongly encouraged to place all protected and public members together within the class definition and reduce the number of public and protected blocks – preferably one each. The use of private within a class is redundant and discouraged.

9.34 Abstract base classes

Define abstract base classes to define interfaces.

Defining abstract base classes to define and enforce an interface. An example of this use is to exploit polymorphic behaviour as is required in design patterns. Abstract base classes must contain a virtual destructor and all methods should be pure virtual.

9.35 Variables

Variables should be declared as close as possible to their first use and must always be initialised.

Variables that are only used within a particular scope shouldn't exist outside that scope. It is possible that the variable cannot be sensibly initialised any sooner within the method. Even if it can be the code is made more readable by only bringing the variable into scope when it is needed.

9.36 Incrementing

Use pre-increment instead of the classical post increment. Pre-increment is generally more efficient than post increment because the compiler does not need to make a copy of the pre-increment value first.

9.37 Commenting classes

Comment classes, member functions and class variables.

A comment describing the class's function should be provided. Each method should also be documented in the header file describing the arguments of the method and their function and the return from that function.

9.38 Operator overloading

Operator overloading is permitted for classes such as iterators, but in general should be avoided unless the semantics are obvious. Don't force the point. An overloaded operator should do exactly what a user expects. If it doesn't – use a named function instead.

Acceptable overloaded operators would be addition and subtraction of modulo integer.

9.38.1 Example

```
class mod_int_t
{
    int const modulus;
    int value;
public:
    operator+=(int i)
    {
        value = (value + i) % modulus;
        return *this;
    }
    mod_int_t(int modulus) :
        modulus(modulus)
    {}
    operator int() const { return value; }
};
```

9.39 No ‘using’ in headers

No “using” in a header. A “using” statement within a header will include that namespace into every file that includes that header, directly or indirectly. Because header files can frequently be deeply nested, it will often be non-obvious that a name conflict arises from a deeply nested header that includes the “using” statement.

10 IDIOMS

An **idiom** (Latin: idioma, "special property", from Greek: ἰδίωμα – idiōma, "special feature, special phrasing, a peculiarity", f. Greek: ἴδιος – ídios, "one's own") is a phrase or a fixed expression that has a figurative, or sometimes literal, meaning. An **idiom's** figurative meaning is different from the literal meaning.

The following coding idioms should be adopted whenever they are applicable. In many cases, idioms are just a matter of convention or preference and in other cases there are good underlying justifications. However, by following the idiomatic approach consistently, the intent of the programming can be more effectively conveyed to future maintainers.

10.1 Forever loops

C++ contains an implicit forever loop construct.

```
for (;;) { /* forever */  
    ...  
}
```

10.2 Simple loops from zero

A program is often required to loop over a range of integers starting from zero and ending at a fixed integer. In many of those cases, the direction of the iteration is unimportant. Summing a range of array values is an example – the result is the same regardless of the order of addition.

```
for (int i(0); i < 10; ++i) { /* iterate from 0 through 9 */  
    ...  
}
```

Consider the following alternative. Although many programmers will initially wonder what is going on, the loop does the exact same thing (in reverse) but with some benefits

```
for (int i(10); i--;) { /* iterate from 9 through 0 */  
    ...  
}
```

- It's shorter
- If the loop terminator (10 in this example) is a complex expression, the latter case evaluates it only once.
- Simple (embedded) compilers often do a better optimisation with the latter and many older 8 bit CPUs (still in use) have decrement and loop support instructions.

10.3 Constant initialisation

The preferred initialisation style for built-in types is as follows.

```
int const SOME_VALUE(10);
```